Unit V: Triggers

Database Triggers

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

What is a Database Trigger?

A Database Trigger is a stored PL/SQL block that automatically executes (fires) in response to specific events in the database (DML modifications, DDL statements, or database system events).

Core Characteristics

- **Implicit Execution:** Fired automatically by the database engine. Cannot be called manually by user queries.
- **Event-Driven:** Bound directly to database events. Fires immediately when the triggering SQL statement executes.
- **Transaction Context:** Runs within the same transaction scope as the DML statement. A failure inside a trigger rolls back the entire transaction.
- **High Risk:** Poorly designed triggers can degrade database performance and cause unexpected side effects (mutating tables).

Trigger Classification

- **DML Triggers:** Bound to INSERT, UPDATE, or DELETE statements on tables or views.
- **DDL Triggers:** Bound to schema modifications (CREATE, ALTER, DROP). Used to audit operations or restrict changes.
- **Database System Triggers:** Bound to database events (STARTUP, SHUTDOWN, SERVERERROR, LOGON, LOGOFF). Ideal for tracking connections.

Row-Level vs. Statement-Level DML Triggers

Understanding the granularity of DML triggers determines how often they fire:

Statement-Level Triggers

• **Execution Frequency:** Fires exactly ONCE per DML statement, regardless of the number of rows affected (even if 0 rows are updated).
• **Syntax:** This is the default trigger behavior. Do not use 'FOR EACH ROW' clause.
• **Variables:** Cannot access individual row values. No access to ':OLD' or ':NEW' record values.
• **Best Use Case:** Enforcing global business restrictions (e.g. prevent table updates outside business hours).

Row-Level Triggers

• **Execution Frequency:** Fires ONCE FOR EACH ROW affected by the DML statement. If a query updates 100 rows, the trigger fires 100 times.
• **Syntax:** Must include the 'FOR EACH ROW' clause in trigger header.
• **Pseudo-records:** Has access to row values:
  - :OLD points to values before query.
  - :NEW points to values after query.
• **Best Use Case:** Auditing cell value changes, generating unique primary keys dynamically.

Trigger Timing and Execution Order

Timings control when the trigger fires relative to the DML event:

Trigger Timings

- **BEFORE Trigger:** Fires before DML execution. Used to validate or alter :NEW values before they are written to disk.
- **AFTER Trigger:** Fires after DML execution. Used to perform post-processing actions (like updating related tables or writing audit records). Cannot modify row values.
- **INSTEAD OF Trigger:** Fires INSTEAD OF the triggering DML. Used primarily on non-updatable database views to route data to baseline tables.
- **Compound Trigger:** Combines BEFORE/AFTER and statement/row timings in a single block, sharing local variables to prevent mutating table errors.

Execution Order Flow

When a statement modifies multiple rows, Oracle executes steps in this sequence:

1. Execute BEFORE statement-level triggers.
2. **For each row affected:**
   a. Execute BEFORE row-level triggers.
   b. Apply DML modifications to the row.
   c. Execute AFTER row-level triggers.
3. Execute AFTER statement-level triggers.

- **Constraints:** Triggers cannot execute transaction commands (COMMIT, ROLLBACK) directly. Trigger size limit is 32KB.

Applications of Triggers

Database triggers automate safety, auditing, and synchronization tasks:

Audit Logging & Tracking

Row-level AFTER triggers capture changes to sensitive columns (e.g. salaries) and write old and new values to log tables with timestamps and username details.

Auto-Generating Primary Keys

BEFORE INSERT row triggers fetch sequence numbers (or UUIDs) and assign them to the :NEW.id field automatically, ensuring unique identifiers.

DDL Change Auditing

DDL triggers track alterations to database tables (e.g. dropping columns) and log schema modifications or prevent changes during release locks.

Data Sync & Replication

Synchronizes replicated tables or denormalized caches automatically when baseline records are altered, keeping data consistent.

THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management